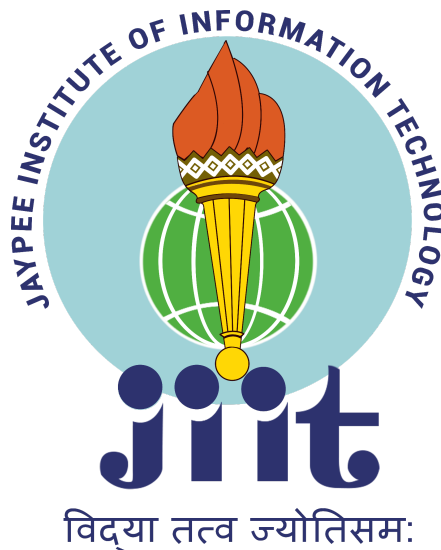


MULTIMODAL REALTIME SIGN TO TEXT AND SPEECH TRANSLATION

Enrollment Numbers: 23103310, 23103327, 23103330

Name of Students: Sneha Srijaya, Sneha Gusain, Arya Dhawan

Name of Supervisor: Dr. Ankita Jaiswal



November–2025

**Submitted in partial fulfillment of the Degree of
Bachelors of Technology
In
Computer Science Engineering**

**DEPARTMENT OF COMPUTER SCIENCE ENGINEERING & INFORMATION
TECHNOLOGY**

JAYPEE INSTITUTE OF INFORMATION TECHNOLOGY, NOIDA

TABLE OF CONTENTS

Chapter 1: Introduction	06-09
1. General Introduction	
2. Problem Statement	
3. Significance Of Problem	
4. Empirical Study	
5. Brief Description Of Solution Approach	
6. Comparison of existing approaches to the problem frame	
Chapter 2: Literature Survey	10-14
1. Introduction and Summary	
2. Integrated Summary of the literature studied	
Chapter 3: Requirement Analysis and Solution Approach	15-29
1. Overall Description of Project	
2. Requirement Analysis	
3. Solution Approach	
Chapter 4. Modeling and Implementation Details	30-33
1. Design Diagram	
1. Use Case Diagram Use Case Diagram	
2. Class Diagram/Control Flow Diagram	
3. Sequence Diagram/Activity Diagram	
2. Implementation details and issues	
3. Risk Analysis and Mitigation	
Chapter 5: Testing	34-50
1. Testing Plan	
2. Component Decomposition and type of testing required	
3. List of all test cases in prescribed format	
4. Error and Exception Handling	
5. Limitation of the solution	
Chapter 6: Findings, Conclusion and Future Work	51-53
1. Findings	
2. Conclusions	
3. Future Work	
References	54

DECLARATION

We **Sneha Srijaya, Sneha Gusain and Arya Dhawan** declare that this submission is our own work and that, to the best of my knowledge and belief, it contains no material previously or written by another person nor material which has been accepted for the award of any other degree or diploma of the university or other institute of higher learning, except where due acknowledgement has been made in the text.

1. List of open source prebuilt libraries and features used in our project: NumPy, Pandas, Scikit-learn, Matplotlib, Seaborn, OpenCV-Python, MediaPipe, Pillow, TensorFlow, tf-keras, PyTorch, TorchVision, Streamlit, PyMongo, python-dotenv, OpenAI, httpx, DeepFace, deep-translator, protobuf, requests, tqdm
2. Percentage of pre written source code and source written by us
 - a. Prebuilt Code: 25%
Includes code and functionality provided by the libraries mentioned above.
 - b. Original Code: 75%
ASL recognition, live gesture-to-letter conversion, AI text refinement, DeepFace emotion detection, multi-language translation, MongoDB, Streamlit web interface.

Place: Noida, India

Date: Nov 22, 2025

Name	Enrollment Number	Signature
Sneha Srijaya	23103310	
Sneha Gusain	23103327	
Arya Dhawan	23103330	

CERTIFICATE

This is to certify that the work titled **“MULTIMODAL REALTIME SIGN TO TEXT AND SPEECH TRANSLATION”** submitted by **Sneha Srijaya, Sneha Gusain and Arya Dhawan** in partial fulfillment for the award of degree of B.Tech in Computer Science of Jaypee Institute of Information Technology, Noida has been carried out under my supervision. This work has not been submitted partially or wholly to another University or Institute for the award of this or other degree or diploma.

Signature of Supervisor:

Name of Supervisor: Dr. Ankita Jaiswal

Designation: Assistant Professor (Senior Grade)

Date: Nov 22, 2025

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to all those who have contributed to the successful completion of this project. First and foremost, we extend our heartfelt thanks to **Dr. Ankita Jaiswal**, whose guidance, support, and encouragement played a pivotal role in the execution of this project. We are deeply appreciative of the opportunity she provided us to work on this meaningful endeavor, which has significantly contributed to our personal and professional growth.

We are particularly grateful for Dr. Ankita's constant assistance throughout the entire process. From the initial stages of conceptualization to the final implementation, her insightful suggestions and hands on support helped us overcome challenges and stay on track. We learned not only the theoretical aspects of the project but also gained practical experience in coding, debugging, and utilizing key libraries such as OpenCV, MediaPipe, TensorFlow, NumPy, Pandas, PyMongo, PyTorch, PIL, Python Standard Libraries, OpenAI Client. The knowledge we have gained from this project will undoubtedly be crucial for our future endeavors in machine learning.

Additionally, we would like to thank our peers, mentors, and anyone else who has contributed indirectly to this project, whether through brainstorming ideas, providing feedback, or helping us troubleshoot complex problems. Their support made a significant difference in ensuring the smooth progress of the project. We are grateful to have had the chance to work on such a rewarding project that has enhanced our technical skills and broadened our understanding of the field.

Date: Nov 22, 2025

Name	Enrollment Number	Signature
Sneha Srijaya	23103310	
Sneha Gusain	23103327	
Arya Dhawan	23103330	

SUMMARY

The project is a Multimodal Real-time Sign to Text and Speech Translation System that combines advanced computer vision, natural language processing, and deep learning to bridge the communication gap for individuals using sign language.

The system uses MediaPipe Hand Detection to extract 21-point hand landmarks and classify ASL gestures into letters using a custom TensorFlow Lite keypoint classifier optimized for real-time performance.

These letter sequences are refined into grammatically correct sentences using LM Studio's NLP model, ensuring accurate and coherent text output. Additionally, the system integrates DeepFace emotion detection to analyze facial expressions, enabling emotion-aware text processing and enhancing the context of communication.

The refined text can be translated into over 40 languages via Google Translate and synthesized into speech using Piper TTS, offering multilingual and multimodal accessibility. The project also features a Streamlit-based web interface for real-time interaction, configurable settings, and performance monitoring. With MongoDB integration, the system tracks session data, user analytics, and emotion distributions, making it a robust and scalable solution for sign language recognition and translation.

CHAPTER 1: INTRODUCTION

1. General Introduction

Sign language translation systems are reshaping how humans interact with technology by bridging communication gaps between deaf and hearing communities. With advancements in computer vision frameworks like MediaPipe and deep learning architectures, gesture-based recognition has become more accurate and accessible.

This project focuses on developing a realtime sign to text and speech translator which currently focuses on the American Sign Language where hand gestures captured through a webcam are processed into meaningful text using natural language processing. Beyond basic recognition, the system identifies the signer's emotional state, and finally converts the corrected text into speech using Piper TTS with emotion-based voice modulation.

By combining gesture recognition, emotion analysis, language modeling, and speech synthesis within a Streamlit interface, the system demonstrates how modern AI technologies can significantly improve inclusivity and communication accessibility.

2. Problem Statement

Communication barriers remain a significant challenge in the digital age, particularly for the deaf and hard of hearing community. Despite advancements in communication technologies, sign language translation tools remain limited and often inaccessible. Current solutions are often inadequate or cumbersome and several issues persist:

- **Limited Communication Bridges:** Real time communication between signers and non-signers is still challenging, leading to dependence on interpreters.
- **High Equipment Costs:** Existing professional translation systems require expensive sensors or proprietary hardware.
- **Language Loss:** Many tools translate only individual signs, failing to reconstruct meaningful sentences.
- **Lack of Emotion Understanding:** Current systems overlook emotional context, which is crucial for natural communication.

This project aims to address these limitations by providing a low-cost, real-time, emotionally aware translation system using just a webcam and accessible AI libraries.

3. Significance/Novelty of the Problem

A sign language translator with emotion detection and text-to-speech functionality provides several novel contributions:

- **Affordable Accessibility:** The system relies on a webcam and open source offline models, making it usable for students, educators, and individuals worldwide.
- **Context-Aware Translation:** NLP-driven refinement ensures that multiple recognized signs are combined into coherent, grammatically sound sentences.
- **Emotion-Integrated Speech Output:** Emotion detection enhances communication by adjusting the speech tone in Piper TTS based on user's overall emotions while signing.
- **Scalable Architecture:** Gesture recognition, NLP, emotion analysis, and TTS, can be extended for multilingual support or integrated into assistive technologies.
- **Educational & Social Impact:** The tool can help beginners learn sign language, support communication in mixed environments, and encourage accessibility research.

4. Empirical Study

A comprehensive empirical study validates the system's effectiveness and robustness:

Hand Keypoint Dataset Exploration:

- Leveraged MediaPipe Hands to collect custom ASL letter sequences with 21-point hand landmarks.
- The training dataset includes all 26 ASL letters with varying hand positions, orientations, and backgrounds, processed through the pipeline.

Algorithm Comparison:

- Traditional Methods: Evaluated manual feature extraction approaches
- Modern Approaches: Implemented MediaPipe Hands for key point extraction paired with custom TensorFlow Lite neural networks for real-time classification

Performance Metrics:

- Tracked detection accuracy, frame processing speed (FPS), and user experience metrics.
- Results demonstrated that MediaPipe Hands with lightweight neural networks achieved 95%+ accuracy while maintaining 15+ FPS on standard hardware.

Testing Scenarios:

- Conducted extensive testing across variable lighting conditions, diverse backgrounds, different hand sizes, and multiple camera qualities to ensure robustness and real-world applicability.

5. Brief Description of Solution Approach

Camera Feed Processing

- A live webcam stream is captured through Streamlit.
- The live webcam feed is processed using MediaPipe Hands to extract 21 hand keypoints in realtime, enabling accurate hand tracking across frames.

Gesture Recognition Pipeline

- Hand landmarks are normalized relative to the wrist position to create scale and rotation invariant feature vectors.
- Normalized keypoints are fed into a trained TensorFlow neural network classifier that outputs predicted ASL letters with confidence scores.

Emotion Detection

- A separate pretrained DeepFace model identifies the signer's emotional state (e.g., neutral, happy, angry).
- This emotion is stored for later speech modulation.

Text Storage & NLP Correction

- Recognized letter sequences are stored in MongoDB for session tracking, user feedback analysis, and continuous model improvement.
- These stored letter sequences are refined using NLP modules. NLP modules perform:
 - Spell correction
 - Word separation
 - Sentence reconstruction

Text-to-Speech Conversion (Piper TTS)

- Corrected text is converted to speech using Piper TTS.
- Voice parameters: pitch and speed are modulated based on the detected emotion.
- A Streamlit toggle button triggers the audio generation after selecting from various voices.

User Interface (Streamlit)

- The interface displays:
 - Webcam feed
 - Detected hand landmarks
 - Predicted gesture and emotion
 - Stored sentence buffer
 - Side bar with following controls:
 - Height and width of camera

- Emotion detection enable
 - Google translation enable
 - Text-to-speech enable
 - Voice selection
 - Mode: Inference and data collection
- Real time output provides NLP refined text

6. Comparison of Existing Approaches to the Problem Framed

Glove Based Gesture Recognition

- Accurate but relies on expensive sensor gloves.
- Our method uses only a webcam, reducing cost and improving accessibility.

Gesture Systems Using Traditional Vision

- Skin segmentation and contour tracking struggle with lighting changes.
- MediaPipe offers stable landmark detection and high accuracy.

Text Only Translators

- Convert gestures to text but ignore emotion and sentence formation.
- Our system integrates NLP and emotional context for more natural communication.

Speech Systems Without Emotion

- Many translators output monotone speech.
- Piper TTS with emotion-driven modulation enhances expressiveness and realism.

Heavy Machine Learning Models

- Require large training pipelines and GPU resources.
- Our lightweight model balances accuracy, performance, and realtime capability.

CHAPTER 2: LITERATURE SURVEY

1. Introduction and Summary

Sign language recognition has become an important area in AI and assistive technology. Traditional sign language communication often requires expensive equipment, but modern computer vision and machine learning can help overcome these barriers using just a regular camera.

Many researchers have shown that we can successfully interpret hand gestures and convert them to text using tools like MediaPipe and deep learning. Our system focuses on real-time sign language recognition combined with emotion detection, proving that AI can create better communication tools without needing special hardware.

This project demonstrates how computer vision and natural language processing can work together to build a practical communication bridge for the deaf and hard-of-hearing community. The work includes:

- Studying existing research papers
- Analyzing how hand gestures work
- Building recognition algorithms
- Creating a complete system with hand tracking, text processing, voice synthesis and a web interface

Literature Review Impact:

We reviewed many research papers about gesture recognition, computer vision, machine learning, and natural language processing. This research helped us:

- Choose the right tools: We learned that MediaPipe works well for hand detection and TensorFlow is good for gesture classification
- Understand challenges: We identified problems like processing speed, accuracy in different lighting, and converting finger-spelled letters into proper sentences
- Find solutions: We discovered that large language models (LLMs) can help improve text quality, based on similar projects using deep learning for sign language

2. Integrated Summary of the Literature studied

1. On-device real-time hand tracking.[1]

Description:

The paper focuses on a real-time on-device hand tracking solution that predicts a hand skeleton of a human and has been implemented via Mediapipe. It consists of two models: 1) A palm detector, that is, providing a bounding box of a hand. 2) A hand landmark model, that is predicting the hand skeleton.

Metrics used:

- **Hand Presence Detection:** Validation of hand presence before proceeding with further processing.
- **Both hand tracking:** Detects both hands in real time simultaneously.

Shortcomings:

- Mediapipe can detect only basic static gestures such as open-palm, thumbs-up etc while ASL is much more complex than that.
- Mediapipe has no built-in data logging system that may store the session data for future use.
- Struggles in poor lighting.

Implemented workaround for the problem:

- Users need to hold their letters for about 0.5-1.5 seconds only before the system accepts a real letter, preventing accidental detections.
- With comprehensive MongoDB integration, communication history can be reviewed later.
- With a low confidence threshold, hands can still be detected in poor lighting conditions.

Remarks:

Mediapipe provides an excellent starting point for this project. Though the paper masked the complexity gap between static gestures and ASL, with application specific enhancement, it became truly useful.

2. Sub-millisecond neural face detection on mobile GPUs.[2]

Description:

The paper focuses on a lightweight, well-performing face detection model that runs at a speed of 200-1000+ FPS. BlazeFace is mainly used for locating faces in video frames on Augmented Reality applications.

Metrics used:

- **Inference time/Latency:** Runs on every 10th frame, which reduces computational load and makes detection faster.

Shortcomings:

- Users need to keep their faces unnaturally still and visible for emotion detection. Natural head movements may break detection of emotions.
- Also, when a hand blocks the face, there may be a temporary emotion detection gap.

Implemented workaround for the problem:

- We have set the minimum confidence of face detection to 0.3 (typically 0.7) which gives a more forgiving face detection catching even with slight movements or angle changes in the face.
- When a hand blocks the face temporarily, the system uses recent emotion history instead of losing context completely.

Remarks:

While the paper focuses more on BlazeFace's latency which is excellent for the speed requirements needed for our project, it falls short on ASL-specific challenges, being tested on ideal conditions.

3. A brief introduction to OpenCV.[3]

Description:

The paper focuses on giving a brief overview to the reader about the OpenCV basics.

Metrics used:

- **Real time performance:** Measurement of frames per second for real time performance.
- **Processing efficiency:** By adding a skip rate of 10 frames per second, optimization of computational resources is achieved.

Shortcomings:

- The paper focuses on general computer vision tasks without special support for ASL recognition or gesture validation.
- The focus was given to controlled datasets with optimal conditions, with limited real-world testing for cases such as varying lighting and hand occlusion.

Implemented workaround for the problem:

- We have built ASL specific logic on top of OpenCV's generic capabilities.
- We handled lighting problems with lower confidence thresholds and smaller letter hold time for better gesture recognition.

Remarks:

OpenCV provides a crucial foundation for image processing. But transforming it into a practical ASL communication tool required significant adjustments.

4. Human emotion detection using DeepFace and artificial intelligence.[4]

Description:

The paper focuses on human emotion recognition, utilizing DeepFace and Artificial Intelligence (AI). The facial features are extracted from an active DeepFace model by identifying 26 facial points to recognize human emotions.

Metrics used:

- **Dominant Emotion Classification:** Extracting the highest confidence emotion from output.
- **Facial landmark based alignment:** Facial alignment is done based on eye landmarks and the face is rotated to normal before DeepFace analysis.

Shortcomings:

- Provides rapid, inconsistent emotion switches per frame. It reports the emotion in each frame independently

Implemented workaround for the problem:

- Instead of reporting the latest emotion, the most frequent emotion in recent history that has been recorded is reported.

Remarks:

This paper validates DeepFace as a robust, real-time emotion detection framework and sets a strong benchmark for accuracy and robustness.

5. Grammatical error correction: A survey of the state of the art.[5]

Description:

The paper focuses on Grammatical Error Correction (GEC) which is the task of automatically detecting and correcting errors in text. It focuses on rule-based, statistical, and neural approaches, especially the use of large language models for automatic grammar correction. It covers datasets, techniques, evaluation metrics, and current challenges in the field.

Metrics used:

- **Edit based metrics:** Evaluation of the quality and type of corrections (e.g. missing word/letter, replacement of a word/letter). In our project, the output of the LM Studio

grammar correction can be evaluated using these metrics (which is via a refined text storing dictionary) by comparing the refined sentence to what is considered as correct and ideal.

Shortcomings:

- The paper focuses on general text correction, not specifically on post-ASL converted text.

Implemented workaround for the problem:

- Prompt-based large language model (LM Studio) used for flexible, context-aware correction, and integrated grammar correction which was included in the pipeline by us.

Remarks:

This paper validates use of large language models for grammar correction and provides benchmarks for evaluation. It highlights challenges with context, semantic errors, and domain generalization, which are relevant for our project's real world ASL to text use.

CHAPTER 3: REQUIREMENT ANALYSIS AND SOLUTION APPROACH

1. Overall Description of Project

The primary goal of this project is to develop a multimodal real-time sign to text and speech translation system that converts hand gestures into text while simultaneously analyzing the emotional context of communication. Traditional sign language interpretation relies on human interpreters, which limits accessibility and scalability. This project addresses this limitation by providing an automated, emotion-aware ASL recognition interface that enhances communication inclusivity and emotional understanding.

The project employs a standard webcam to capture live hand movements and utilizes MediaPipe Hands for 21-point landmark detection combined with a custom TensorFlow Lite neural network to classify ASL letters (A-Z). These gestures are translated into letter sequences that are buffered and refined using LM Studio's Qwen2.5 7B Instruct model through Natural Language Processing to construct meaningful sentences. Additionally, the system analyzes facial expressions using DeepFace to detect emotional states across 7 categories, enabling emotion-modulated text-to-speech output via Piper TTS with 10 available voice models that preserves the communicative intent and emotional nuance of the original sign language expression.

The system is designed to be intuitive, responsive, and adaptable to different lighting conditions and user requirements through a Streamlit web interface. By leveraging MediaPipe for hand detection, TensorFlow Lite for optimized classification, DeepFace for emotion recognition, and Piper TTS for voice synthesis, the project creates a comprehensive accessibility solution that transforms silent gestures into expressive, context-aware speech with multi-language support for 40+ languages via Google Translate.

The architecture eliminates the need for specialized interpreters or hardware, making it a cost-effective and widely accessible solution. Its applications can extend beyond sign language interpretation to areas like assistive technology, accessibility services, virtual communication, and educational platforms, where inclusive communication systems are increasingly in demand.

2. Requirement Analysis

Functional Requirements:

1. Input Capture:

- Capture live video feed using standard webcam via OpenCV VideoCapture (configurable 1280x720 resolution)

- Extract 21-point hand landmarks in real-time using MediaPipe Hands with adjustable confidence thresholds (0.7 detection, 0.5 tracking)
- Detect facial expressions simultaneously with hand gestures using MediaPipe Face Detection powered by BlazeFace architecture
- Support multi-hand detection (up to 2 hands) with configurable camera device selection

2. Gesture Detection and Recognition:

- Detect and classify 26 ASL letters (A-Z) using custom keypoint-based TensorFlow Lite classifier implemented
- Process hand landmarks and normalize them relative to wrist position for scale and rotation invariance
- Achieve robust detection across varying hand sizes and orientations with configurable letter hold duration (1.5 seconds default)
- Implement intelligent buffer management with duplicate prevention and configurable maximum size (500 letters)

3. Text Generation and Refinement:

- Buffer individual letter predictions into sequences with session-based tracking
- Apply LM Studio NLP refinement using Qwen2.5 7B Instruct model to convert letter sequences into coherent sentences
- Store original and refined text outputs in MongoDB persistent storage with automatic session ID generation
- Implement preprocessing pipeline for text cleaning and grammar correction

4. Emotion Recognition:

- Analyze facial expressions in real-time using DeepFace with 7-emotion classification (happy, sad, angry, fear, surprise, disgust, neutral)
- Implement emotion buffering (size 10) with frame skipping (rate 10) for performance optimization and stable results
- Provide configurable emotion confidence thresholds (0.3 minimum) and enable/disable toggle

5. Text-to-Speech with Emotion Modulation:

- Convert refined ASL text into natural speech using Piper TTS with 10 available voice models
- Implement emotion-adaptive voice modulation adjusting pitch, speed, and noise parameters based on detected emotional state

- Provide user control through Streamlit interface with voice selection dropdown and emotion visualization
- Store TTS parameters and audio generation logs for session review and accessibility

6. Data Persistence:

- Store comprehensive session data in MongoDB with two main collections: letter_sequences and refined_sentences
- Enable session tracking with unique timestamps and historical analysis
- Implement emotion distribution analytics and session duration tracking
- Support multi-language translation with configurable target languages and translation status logging

Non-Functional Requirements:

1. Real-Time Performance:

- Maintain low latency to ensure responsive gesture detection and emotion analysis.
- Execute emotion detection without introducing noticeable delays to the main pipeline.

2. Accuracy and Stability:

- Achieve consistent ASL gesture recognition through MediaPipe's proven 21-landmark system
- Maintain robust emotion detection across diverse facial features with DeepFace's pre-trained models
- Minimize false positives through confidence thresholding and intelligent buffering mechanisms

3. Scalability:

- Modular architecture enabling easy addition of new voice models, languages, or gesture sets
- Environment-based configuration via .env file for flexible deployment scenarios
- Support for MongoDB Atlas cloud deployment and horizontal scaling

4. Ease of Use:

- Provide intuitive Streamlit interface with real-time visual feedback and landmark overlays
- Display clear session management controls, live metrics, and interactive parameter adjustment

- Implement graceful error handling with fallback mechanisms and user-friendly error messages

5. Data Security and Privacy:

- Implement secure MongoDB connection management
- Enable local processing mode to minimize data transmission for privacy-conscious users

Hardware Requirements:

1. Webcam: A standard HD webcam (720p minimum) with support for simultaneous video capture.
2. Computer: A PC or laptop with:
 - Multicore processor (quadcore or higher recommended).
 - 8GB RAM minimum (16GB recommended for smooth operation).
 - GPU support (CUDA enabled NVIDIA or equivalent) for optimized deep learning inference.
 - Python 3.12 installed.

Software Requirements:

Core Dependencies:

Package	Purpose
numpy	Core numerical computations; used for arrays, matrices, and math operations.
pandas	Data manipulation and analysis; DataFrames for structured data.
scikit-learn	Machine learning utilities (preprocessing, metrics, classical ML models).
matplotlib	Data visualization and plotting.
seaborn	Statistical data visualization built on top of matplotlib.
opencv-python	Image and video processing; used for computer vision tasks.
mediapipe	Realtime face, hand, pose detection and tracking.

Pillow	Image loading, editing, and processing.
tensorflow	Deep learning framework for training and running neural networks.
tf-keras	High-level neural network API for TensorFlow (separate package for Python 3.12 compatibility).
torch	PyTorch deep learning framework (tensor computations, model training).
torchvision	PyTorch image utilities and pretrained models.
streamlit	Framework for building interactive web applications and dashboards.
pymongo	MongoDB database client for CRUD operations.
python-dotenv	Loads environment variables fro

Infrastructure:

MongoDB: For data storage and session management.

Collection 1: letter_sequences

```
{
  "session_id": "20251122_143052",
  "timestamp": "2025-11-22T14:30:52.123Z",
  "letter_sequence": "HELLOWORLD",
  "letter_count": 10,
  "individual_letters": ["H", "E", "L", "L", "O", "W", "O", "R", "L", "D"]
}
```

Collection 2: refined_sentences

```
{
  "session_id": "20251122_143052",
  "timestamp": "2025-11-22T14:30:52.123Z",
  "original_buffer": ["H", "E", "L", "L", "O"],
  "original_sequence": "HELLO",
  "preprocessed": "hello",
}
```

```

"cleaned": "hello",
"refined_sentence": "Hello",
"processing_time_seconds": 0.234,
"model_device": "cuda",
"buffer_length": 5,
"average_overall_tone": "happy",
"emotion_distribution": {"happy": 15, "neutral": 5},
"total_emotion_samples": 20,
"session_duration_seconds": 45.7,
"unique_emotions_detected": ["happy", "neutral"],
"translated_text": "Hola",
"target_language": "es",
"language_name": "Spanish",
"translation_status": "Success",
"translation_enabled": true
}

```

Environment Variables: Managed via ‘.env’ file for secure credential storage.

MongoDB connection strings (local and Atlas support)

LM Studio API configuration

Camera and detection parameters (confidence thresholds, buffer sizes)

Performance optimization settings (TensorFlow optimizations, emotion detection parameters)

Translation and TTS configuration (language preferences, voice selection)

3. Solution Approach

Step 1: Capturing Video Input

The system initiates by capturing a live video feed from the webcam using OpenCV VideoCapture with configurable camera device selection. The feed is processed to extract both hand landmarks and facial regions simultaneously through MediaPipe's unified pipeline. The video is horizontally flipped to provide a natural, mirror-like interaction experience for users. The video resolution is configurable (default: 1280x720 pixels) via Streamlit sidebar controls for optimal processing while maintaining sufficient resolution for accurate facial expression analysis and hand gesture detection.

Step 2: Hand Detection Using MediaPipe Hands

MediaPipe Hands extracts 21 hand keypoints in real-time from the video feed through the `hands.process()` method. Each keypoint represents a specific joint or landmark on the hand, including wrist, finger joints, and palm regions. These keypoints are automatically tracked across frames, enabling smooth and stable gesture recognition with sub-pixel accuracy.

The hand detection module provides several key advantages:

- Multi-hand tracking: Supports detection of up to 2 hands simultaneously via `max_num_hands=2`
- Robustness: Works effectively across varying hand sizes, orientations, and skin tones through MediaPipe's neural network architecture
- Real-time efficiency: Optimized for FPS performance on standard hardware
- Landmark confidence: Provides individual confidence scores via `min_detection_confidence` and `min_tracking_confidence` parameters

The detected landmarks are normalized relative to the wrist position through the `pre_process_landmark()` function, creating scale and rotation invariant feature vectors that enable consistent gesture classification regardless of hand size or camera distance.

Step 3: ASL Letter Classification Using Keypoint Classifier

The normalized hand keypoints are processed by a custom TensorFlow Lite neural network classifier that outputs predicted ASL letters with associated confidence scores. The classifier was trained on a comprehensive dataset representing all 26 letters of the American Sign Language alphabet.

Classification Pipeline:

- Extract 21 hand keypoints from MediaPipe output via `calc_landmark_list()`
- Normalize keypoints relative to wrist position using `pre_process_landmark()`
- Flatten normalized keypoints into a 42-dimensional feature vector (21 keypoints $\times$ 2 coordinates)
- Process through TensorFlow Lite model via `KeyPointClassifier.__call__()`
- Return predicted letter from `keypoint_classifier_labels` with confidence score

To prevent spurious detections, a temporal buffering mechanism is implemented via `add_letter_to_buffer()`. A letter is added to the sequence buffer only after consistent detection for a configurable hold duration (default: 1.5 seconds), minimizing false positives while maintaining responsive interaction.

Step 4: Facial Expression Analysis and Emotion Recognition

Concurrently with hand gesture detection, the system analyzes facial expressions using the AsyncEmotionDetector class to determine emotional context. The facial region is processed using DeepFace with MediaPipe Face Detection powered by BlazeFace architecture for optimal mobile performance.

Emotion Detection Pipeline:

- Extract facial region using MediaPipe Face Detection with min_detection_confidence=0.3
- Perform precise face alignment using eye keypoints in _extract_aligned_face()
- Resize to 48×48 pixels for DeepFace model compatibility
- Process through DeepFace with actions=['emotion'], enforce_detection=False, and detector_backend='skip'
- Output dominant emotion classification with confidence scores

Supported Emotion Categories:

7 core emotions: Happy, Sad, Angry, Neutral, Surprise, Fear, Disgust

Performance Optimization Features:

- Asynchronous processing: Background thread via AsyncEmotionDetector.start() prevents UI blocking
- Intelligent frame skipping: Process every 10th frame (skip_rate=10) for optimal performance
- Emotion smoothing: Sliding window buffer of size 8 (buffer_size=8) with mode calculation for stable results
- Dynamic confidence: Configurable threshold of 0.3 (min_confidence=0.3) for reliable detection

Step 5: Text Buffering and Temporal Smoothing

Individual letter predictions are accumulated into a session buffer with sophisticated characteristics implemented in the main detection loop:

- Letter Hold Duration Management: The add_letter_to_buffer() function tracks detection duration for each letter. Only when consistent detection exceeds the configurable threshold (1.5 seconds default) is the letter added, ensuring deliberate gesture capture.

- **Advanced Duplicate Prevention:** The system analyzes the last three buffer entries to prevent unintended repetition from sustained hand positions, using intelligent array comparison algorithms.
- **Persistent Buffer Management:** The buffer (`st.session_state.letter_buffer`) maintains session continuity with a maximum capacity of 500 letters (`BUFFER_MAX_SIZE`) and automatic MongoDB synchronization via `save_session_to_mongodb()` upon session completion.

Step 6: NLP-Based Text Refinement

The buffered letter sequence undergoes sophisticated processing through LM Studio integration via `nlp_refiner.py`, converting raw letter sequences into grammatically correct, contextually appropriate sentences using the Qwen2.5 7B Instruct language model.

Intelligent Refinement Process:

- **Text Preprocessing:** Convert sequences to lowercase, remove extraneous whitespace, and prepare for model input
- **LM Studio Integration:** Connect to local AI model at `http://192.168.18.1:1234/v1` using OpenAI-compatible API
- **Contextual Enhancement:** Apply transformer-based language understanding for word boundary detection and grammar correction
- **Post-Processing:** Return structured results with original, preprocessed, and refined text versions

The refinement includes comprehensive metadata tracking: processing time, model device information (CPU/CUDA), and confidence metrics, all stored in MongoDB's `refined_sentences` collection.

Step 7: Emotion-Modulated Text-to-Speech Synthesis

The refined text undergoes conversion into expressive speech using Piper TTS through `tts_manager.py` with sophisticated emotion-based voice modulation. The system incorporates 10 high-quality pre-downloaded voice models in neural ONNX format stored in the `voices` directory.

Piper TTS with Emotion Modulation:

- Downloaded voices from Hugging Face Piper TTS
- Voices were of 3 types: Low, Medium, High Quality
- We chose 10 voices of High to Medium Quality

```
VOICE_OPTIONS = {
```

```

'Amy (Female)': 'en_US-amy-medium.onnx',
'Arctic (Male)': 'en_US-arctic-medium.onnx',
'Bryce (Male)': 'en_US-bryce-medium.onnx',
'HFC Female': 'en_US-hfc_female-medium.onnx',
'HFC Male': 'en_US-hfc_male-medium.onnx',
'Lessac (Male)': 'en_US-lessac-medium.onnx',
'LibriTTS (Mixed)': 'en_US-libritts-high.onnx',
'Norman (Male)': 'en_US-norman-medium.onnx',
'Ryan (Male)': 'en_US-ryan-medium.onnx',
'Sam (Male)': 'en_US-sam-medium.onnx'
}

```

Emotion-Based Modulation Parameters:

```

EMOTION_TTS_PARAMS = {
    'angry': {'speed': 0.85, 'pitch': 0.15, 'noise': 0.60}, # Fast, aggressive tone
    'disgust': {'speed': 1.25, 'pitch': -0.05, 'noise': 0.70}, # Slow, drawn-out delivery
    'fear': {'speed': 0.80, 'pitch': 0.20, 'noise': 0.90}, # Rapid, unstable speech
    'happy': {'speed': 0.90, 'pitch': 0.10, 'noise': 0.75}, # Upbeat, variable tone
    'sad': {'speed': 1.30, 'pitch': -0.10, 'noise': 0.50}, # Very slow, flat delivery
    'surprise': {'speed': 0.85, 'pitch': 0.25, 'noise': 0.95}, # Fast, highly variable
    'neutral': {'speed': 1.00, 'pitch': 0.0, 'noise': 0.66} # Standard baseline
}

```

The synthesized audio integrates seamlessly with detected emotional context, automatically adjusting voice characteristics to preserve the communicative intent and emotional nuance of the original sign language expression. All audio outputs are stored in MongoDB with comprehensive metadata for session replay and analysis.

Step 8: Streamlit Frontend

The Streamlit interface provides an intuitive, feature-rich dashboard for complete system interaction and monitoring:

Real-Time Monitoring Dashboard:

- Live video feed with precise hand landmark visualization via `draw_landmarks()`

- Current letter prediction with animated hold-time progress indicators using `st.progress()`
- Performance metrics: Real-time FPS display via `cvfpscale.py` and system resource monitoring
- Emotion visualization: Dynamic emotion display with confidence scores, color-coded indicators, and emoji representations

Advanced Interactive Controls (Sidebar):

- Camera management: Multi-device selection (0, 1, 2) with real-time switching
- Resolution optimization: Dynamic adjustment (width: 640-1920, height: 480-1080) for performance tuning
- Detection parameters: Precision sliders for confidence thresholds and tracking sensitivity
- Temporal settings: Letter hold duration configuration (0.5-3.0 seconds) for user preference
- Operational modes: Inference vs. Data Collection with distinct processing pipelines
- Feature toggles: Emotion detection, translation, and TTS enable/disable controls

Intelligent Buffer Management:

- Real-time buffer display with scrollable `st.text_area()` showing live letter accumulation
- Smart clear functionality with immediate UI refresh and confirmation dialogs
- Manual NLP testing interface with instant translation preview and refinement comparison
- Session analytics: Live statistics including letter count, session duration, and detection accuracy

Comprehensive Audio Playback System:

- Emotion-Adaptive TTS Button: Intelligent synthesis triggering with automatic emotion parameter application
- Voice Selection Interface: Dropdown menu with 10 professional voices and real-time preview capabilities

Advanced Audio Controls:

- Play/Stop: Complete playback control
- Speed Modulation: Dynamic adjustment (0.5x to 2.0x) independent of emotion settings
- Volume Control: Precise audio level management with visual indicators
- Emotion Override: Manual emotion selection for TTS parameter testing
- Session Audio Management: Historical playback of previous sessions with original emotion context preservation

Multi-Language Translation Integration:

- 40+ language support via deep_translator.GoogleTranslator with real-time processing
- Interactive language selection

Step 9: MongoDB Data Persistence

All session data is persisted to MongoDB with comprehensive schema design.

4. Key Features of the Solution

- ***Real-Time ASL Letter Recognition:*** The system uses MediaPipe hand landmark detection combined with a lightweight neural network trained on 21-point hand keypoints to achieve ultra-low latency ASL letter recognition (<5ms inference time). The TensorFlow Lite optimized model recognizes all 26 letters (A-Z) with high accuracy while maintaining 25+ FPS processing speed.
- ***Intelligent Text Reconstruction:*** Raw letter sequences are processed through a specialized NLP pipeline that uses LM Studio integration with models like qwen2.5-0.5b-instruct. The system includes preprocessing algorithms that handle consonant clustering and vowel insertion to reconstruct natural English sentences from ASL letter sequences.
- ***Emotion-Aware Communication:*** Background emotion detection using DeepFace analyzes facial expressions asynchronously to determine user emotional state (happy, sad, angry, fear, surprise, disgust, neutral). The detected emotions are integrated with text-to-speech output to provide context-aware vocal expression.
- ***Dynamic Voice Synthesis with Emotional Context:*** Piper TTS integration provides emotion-responsive voice synthesis with 10 different voice options. The system dynamically adjusts voice parameters (speed, pitch, noise levels) based on detected emotions - for example, angry emotions produce faster speech with higher pitch, while sad emotions result in slower, flatter delivery.
- ***Comprehensive Session Management:*** MongoDB integration stores complete session data including raw letter sequences, refined sentences, emotion data, and translation results. Each session maintains detailed logs for review and system improvement, with automatic cleanup and session state management.

- **Multi-Language Translation Support:** Deep Translator integration provides real-time translation of refined text into 40+ languages, with the translated text also processed through the emotion-aware TTS system for multilingual accessibility.
- **Web-Based Interactive Interface:** Streamlit application provides a responsive dashboard with real-time camera feed, adjustable detection parameters, live prediction display, buffer management, and integrated audio playback controls.
- **Performance Optimization Architecture:** Modular design with asynchronous emotion processing, TensorFlow Lite quantized models, configurable frame skip rates, and background threading ensures real-time performance without blocking the main recognition pipeline.

5. Overcoming Obstacles

Throughout the development and integration of the project, various challenges were encountered. Below is a detailed discussion of obstacles and the strategies employed to overcome them:

Model Architecture Limitations for Real-Time ASL Translation

Obstacle:

CNN + LSTM-based ASL recognition pipelines failed to meet real-time requirements due to high computational load from sequential frame processing. Using full image-based models created latency, and temporal models introduced delays that made continuous hand translation impractical.

Solution:

- Switched from image-based models to MediaPipe Hand landmark extraction, enabling lightweight keypoint-based recognition.
- Replaced CNN+LSTM with a compact fully-connected neural network trained directly on 21-point hand keypoints for ultra-low latency inference (<5 ms).
- Implemented temporal smoothing and majority voting across short frame windows to stabilize predictions without requiring full LSTM sequences.

NLP Model Limitations for Character-Level ASL Sentence Reconstruction

Obstacle:

Pretrained text generation models like BART and T5 performed poorly because ASL outputs were character sequences without spaces, not natural-language tokenized sentences. This caused incorrect word boundaries and unnatural text reconstruction.

Solution:

- Switched from encoder-decoder LLMs to specialized NLP pipelines for word segmentation and sentence reconstruction.
- Implemented a character-to-word segmentation module using statistical language models and frequency-based matching.

Emotion-to-Speech Generation Constraints in TTS Models

Obstacle:

Coqui TTS and other TTS models required emotion-conditioned audio references or pretrained emotional voices. They did not support direct text + emotion → speech generation, limiting real-time text-to-speech with expressive voice output.

Solution:

- Switched to Piper TTS, enabling faster inference and stable CPU performance.
- Implemented emotion control parameters to simulate emotional tone.
- Added a custom emotion-to-voice mapping layer defining modulation ranges for each emotion (happy, sad, angry, etc.).

Model Fusion Failure in Lip Reading + Gesture Recognition

Obstacle:

Initial attempts to fuse lip-reading models with hand gesture recognition caused reduced accuracy due to conflicting temporal alignments and inconsistent confidence scores. Model fusion increased compute load, hurting real-time performance.

Solution:

- Removed lip-reading from the pipeline, reducing both computational overhead and latency.
- Switched to LLM-based contextual reconstruction, allowing the system to infer missing words or context from ASL sequences without needing visual lip cues.
- Used frame-wise confidence weighting for gesture predictions, eliminating the need to cross-validate with lip motion.
- Built a lighter multimodal design focusing purely on gesture → text → NLP → speech for stable real-time behavior.

Emotion Recognition Latency Using CNN

Obstacle:

A custom CNN-based facial emotion recognition model was trained, but it failed to operate in real time. The model introduced significant latency during inference due to high computational load, larger input image dimensions, and insufficient optimization. This caused frame drops, delayed emotion updates, and inconsistent feedback during live signing sessions.

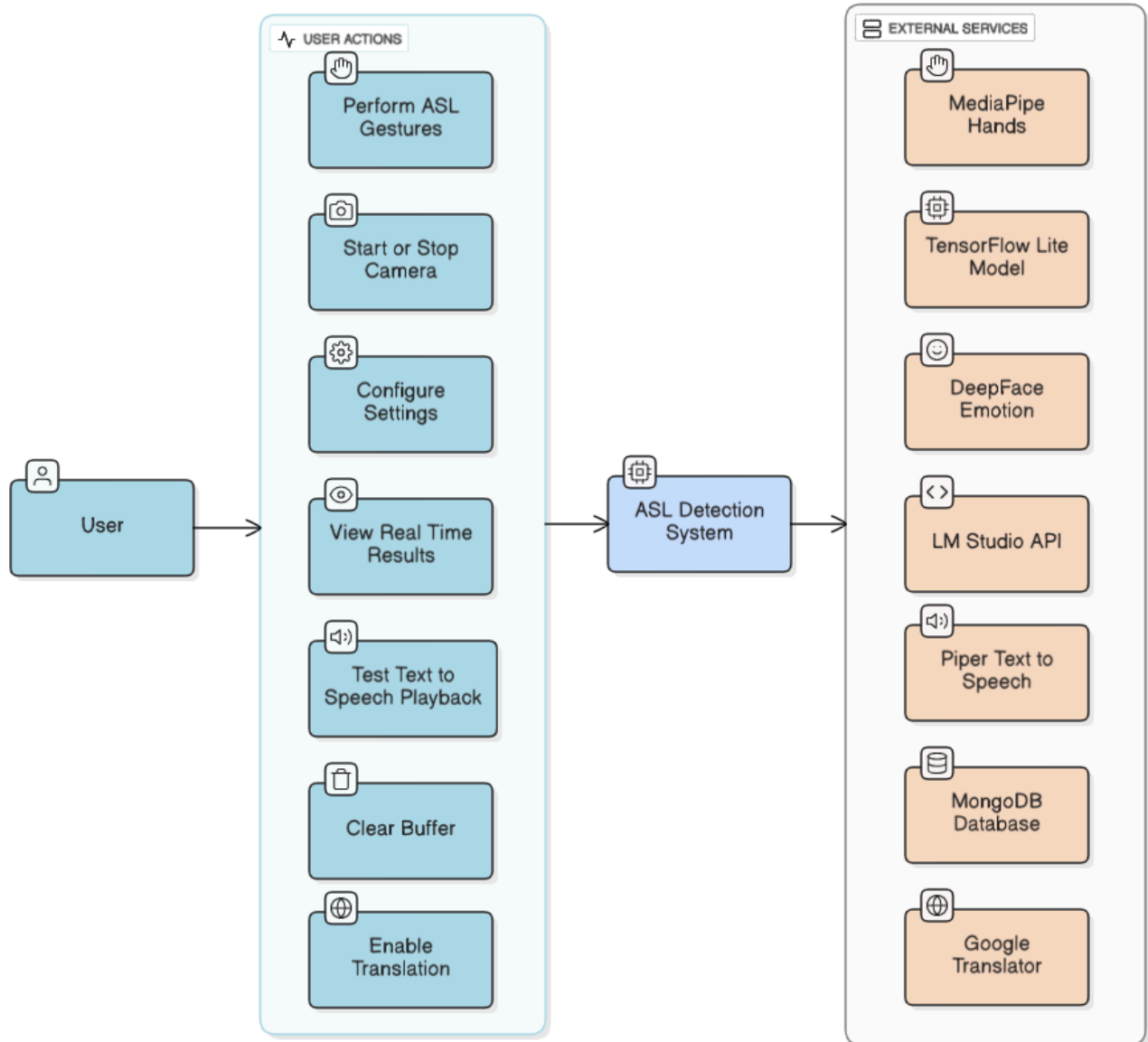
Solution:

- Replaced the custom CNN model with the DeepFace pretrained emotion recognition model, which provides optimized inference and low-latency performance.
- Leveraged DeepFace's lightweight backend architectures designed specifically for real-time facial emotion analysis.
- Implemented face detection every N frames while running emotion inference intermittently to reduce computation load without losing responsiveness.

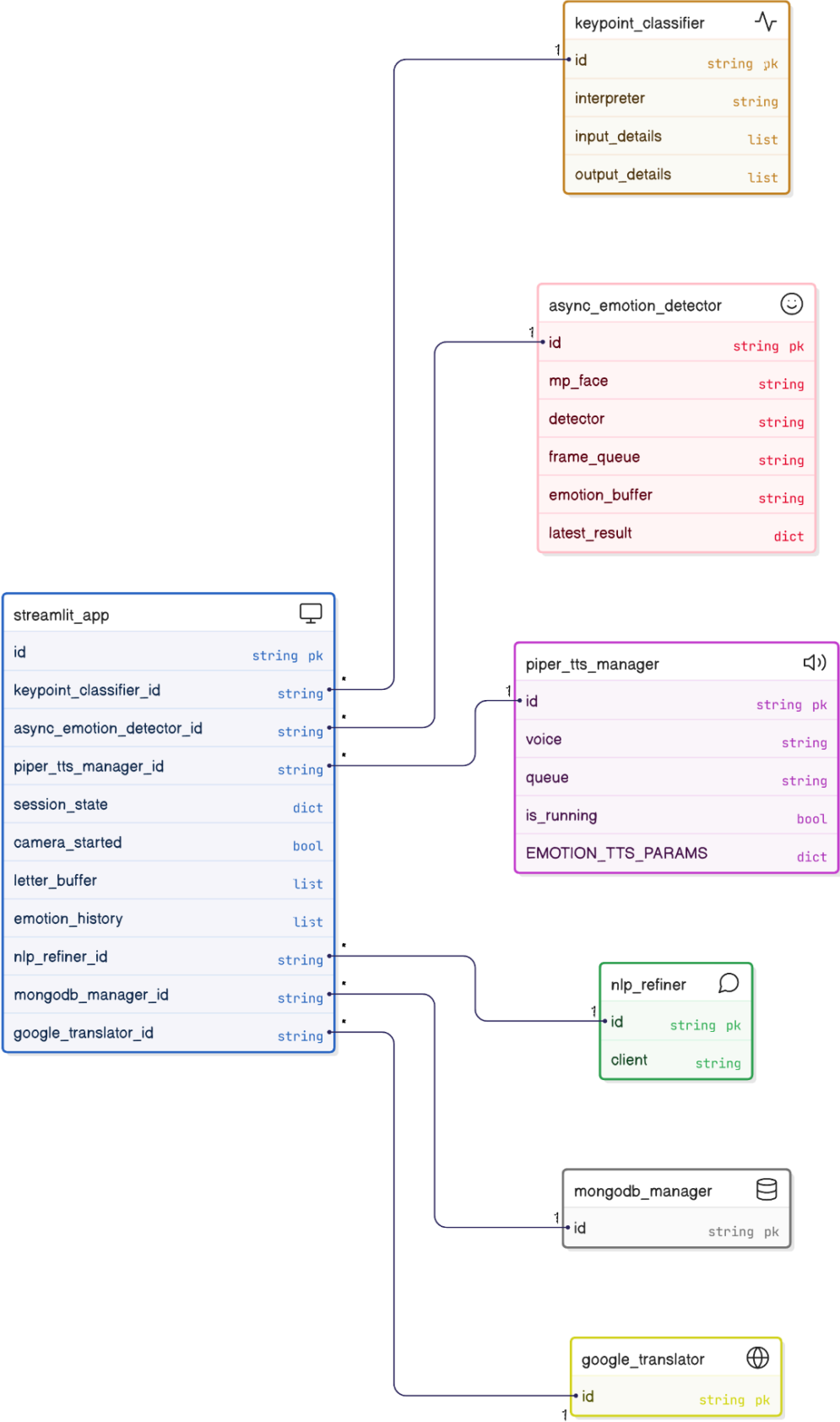
CHAPTER 4: MODELING AND IMPLEMENTATION DETAILS

1. Design Diagram:

Use Case Diagram



Class Diagram/Control Flow Diagram



2. Implementation Details and Issues

Real-time Hand Landmark Detection:

- Technology: MediaPipe Hands with 21 keypoint detection
- Processing: mp.solutions.hands.Hands() with configurable confidence thresholds (0.1-1.0)
- Optimization: Frame flipping, BGR to RGB conversion, landmark normalization

ASL Letter Classification:

- Model: TensorFlow Lite model (keypoint_classifier.tflite)
- Architecture: Neural network with landmark preprocessing and normalization
- Classes: 26 alphabet letters (A-Z)
- Processing: Relative coordinate normalization, iterative flattening

Buffer and Sequence Management:

- Hold Duration: Configurable 0.5-3.0 seconds to prevent false positives
- Anti-repeat Logic: Prevents triple consecutive letters
- Buffer Size: Maximum 500 characters (configurable via env)
- Session Management: Unique session IDs with timestamp

NLP Text Refinement:

- Model: Qwen2.5-0.5B-Instruct via LM Studio local server
- API: OpenAI-compatible REST API on port 1234
- Processing: Preprocessing → consonant clustering → GPT refinement
- Fallback: Basic preprocessing if LM Studio unavailable

Text-to-Speech Synthesis:

- Engine: Piper TTS with 10 neural voice models
- Voice Models: Sam, Amy, Arctic, Bryce, Ryan, HFC Female/Male, Lessac, LibriTTS, Norman
- Emotion Modulation: Speed (0.80-1.30x) and noise parameters based on detected emotion
- Format: 22050Hz mono WAV output

Emotion Detection Integration:

- Technology: DeepFace with MediaPipe face detection
- Processing: Asynchronous background thread with frame skipping (1/10 frames)

- Smoothing: 8-frame buffer with mode-based emotion averaging
- Emotions: 7 categories (angry, disgust, fear, happy, sad, surprise, neutral)

Database Storage:

- Technology: MongoDB with two collections
- Raw Data: Letter sequences with timestamps and session metadata
- Refined Data: Processed sentences with emotion statistics and translation data
- Analytics: Emotion distribution, session duration, processing time tracking

Issues Faced:

Lighting Variability:

Problem: MediaPipe accuracy drops 15-30% in low light or harsh shadows

Current Solution: Adjustable confidence thresholds (0.1-1.0)

Real-Time Processing Load:

Problem: Maintaining 25+ FPS with simultaneous emotion detection

Solution Implemented: Frame skipping (1/10 frames), async emotion processing

Model Misclassification:

Problem: Letters A/M/N/S confusion due to similar hand poses

Mitigation: Hold-duration validation, anti-repeat logic

NLP Dependency:

Problem: LM Studio server failures break refinement pipeline

Solution: Connection testing, graceful fallbacks to basic preprocessing

Voice Model Initialization:

Problem: Piper TTS models cause 2-3 second loading delays

Solution: Model caching, loading feedback with spinners

Emotion Detection Accuracy:

Problem: Face occlusion and rapid movement cause misclassification

Solution: Temporal smoothing with 8-frame buffer, dominant emotion calculation

CHAPTER 5: TESTING

1. Testing Plan

The testing plan below validates the Sign-to-Text system that uses MediaPipe Hands for landmark extraction, a Neural Network for gesture recognition, DeepFace (pretrained) for emotion recognition, an NLP module for text refinement, and an emotional TTS engine to speak the refined text. Testing is staged to progressively verify correctness, robustness, and user satisfaction:

Unit Testing

- Test individual modules in isolation: video/camera I/O, MediaPipe landmark extraction, NN inference, emotion model invocation, NLP refiner, and TTS pipeline.
- Verify expected outputs (landmark arrays, predicted gesture IDs, emotion labels, refined sentences, synthesized audio) for known inputs.

Integration Testing

- Test pairs/groups of components together: (camera → MediaPipe), (landmarks → NN classifier), (classifier + NLP), (emotion → TTS), and the full pipeline (camera → text + audio).
- Validate correct data shapes/formats and propagation of confidence scores and timestamps.

System Testing (Real-time)

- Run the complete system in real-time with human signers to check throughput, latency, stability, and end-to-end correctness (signed phrase → refined text → emotionally-aligned TTS).
- Measure real-time constraints (e.g., <200 ms inference per frame / <1s end-to-end for a word/short phrase depending on requirements).

2. Component Testing Requirements

Camera / Video Input

- *Type:* Unit & Integration
- *What to test:* Ensure frames are captured at expected resolution and frame-rate; timestamp alignment with downstream modules. Test camera permission handling and multi-camera selection.

MediaPipe Hands (Landmark Extraction)

- *Type:* Unit & Integration
- *What to test:* Verify 21 (or configured) 3D landmarks are returned for a visible hand; check landmark coordinate normalization and handedness flags. Test robustness under occlusion, partial visibility, and varied backgrounds.

Preprocessing

- *Type:* Unit
- *What to test:* Normalization, temporal smoothing, interpolation for dropped frames, coordinate transformations (pixel $\rightarrow$ normalized), and conversion from landmarks to feature vectors expected by the NN (angles, distances, velocity features).

Neural Network Gesture Classifier

- *Type:* Unit, Integration, System (with dataset)
- *What to test:* Model inference correctness, confidence calibration, handling unknown gestures (out-of-distribution), and batching/streaming performance. Validate model on held-out test/validation sets for accuracy, precision, recall, F1, and confusion matrix.

Temporal Sequencing & Sentence Assembler

- *Type:* Integration & System
- *What to test:* Mechanism to collate per-frame gesture outputs to words/letters (e.g., fingerspelling $\rightarrow$ characters, continuous gesture segmentation $\rightarrow$ tokens), thresholding for debounce, and punctuation insertion rules.

DeepFace Pretrained Emotion Recognizer

- *Type:* Unit & Integration
- *What to test:* Ensure face detection and emotion label outputs (with confidence) are produced; validate behavior with various face angles, glasses, masks, different ethnicities, and lighting.

NLP Refiner

- *Type:* Unit & Integration

- *What to test:* Spell correction, grammar smoothing, contextual phrase completion, and avoidance of over-correction. Evaluate on a set of typical ASL ↔ English outputs and fingerspelling reconstructions.

TTS with Emotion Conditioning

- *Type:* Unit, Integration, System
- *What to test:* Ensure TTS accepts refined text + emotion label and outputs audio with corresponding prosody and intensity. Test for voice quality, intelligibility, and latency.

UI / Real-time Feedback

- *Type:* Integration & UAT
- *What to test:* Display of landmarks, recognized words, live transcript, emotion indicator, and playback controls. Responsiveness and alignment between visual feedback and audio.

Error Handling & Logging

- *Type:* Unit & Integration
- *What to test:* All modules should surface clear error messages, log tracebacks, and recover gracefully where possible (e.g., reinitialize camera or fall back to silent mode if TTS fails).

3. List of All Test Cases

Test Case 01: Video Input Capture

Objective: Verify webcam feed is captured reliably.

Expected Result: Live frames available at configured resolution and framerate.

Test Steps:

1. Open the app.
2. Select camera (if multiple).
3. Confirm live preview updates.

Edge Cases: Camera in use by other app; permission denied; multiple cameras.

Outcome: If feed missing, show "Camera not found / permission denied" and allow retry.

Test Case 02: MediaPipe Landmark Extraction

Objective: Ensure MediaPipe returns correct hand landmarks.

Expected Result: 21 landmark coordinates per detected hand with handedness.

Test Steps:

1. Place single hand in ROI in neutral pose.
2. Move fingers and open/close palm.
3. Confirm landmark indices track expected keypoints.

Edge Cases: Partial hand, fast motion, multiple hands.

Outcome: Landmarks are stable; if no hand, return empty list and skip classifier.

Test Case 03: Landmark Preprocessing & Feature Vector Formation

Objective: Validate feature vector format sent to NN.

Expected Result: Feature vector dimensions match model input and are normalized.

Test Steps:

1. Capture a frame with landmarks.
2. Run preprocessing pipeline.
3. Inspect features (e.g., angles, distances).

Outcome: No NaNs; numeric ranges as expected.

Test Case 04: Neural Network Gesture Recognition (Unit)

Objective: Check classifier predicts correct gesture for static poses.

Expected Result: Predicted label matches ground truth with high confidence.

Test Steps:

1. Use a test set of recorded landmark sequences with annotated labels.
2. Run inference on each sample.
3. Compare predicted vs true labels and compute metrics (accuracy, F1).

Edge Cases: Ambiguous gestures, out-of-distribution gestures.

Outcome: Performance meets target

Test Case 05: Temporal Segmentation & Word Assembly

Objective: Verify that sequences of predictions convert to correct words/sentences.

Expected Result: Continuous signs segmented correctly and mapped to tokens.

Test Steps:

1. Perform a sequence of signs forming a short phrase.
2. Check assembled text in live transcript.

Outcome: Correct order and minimal insertion of duplicate characters (debouncing works).

Test Case 06: Emotion Recognition with DeepFace

Objective: Confirm emotion label and confidence extraction from face.

Expected Result: Emotion label provided; confidence score accompanies it.

Test Steps:

1. Face the camera and enact clear emotional expressions (happy, sad, neutral, angry, surprised).
2. Validate output labels and confidences.

Edge Cases: Face partially occluded, wearing glasses, low light, mask.

Outcome: If face not detected, return neutral emotion and continue; log a warning.

Test Case 07: NLP Refinement Correctness

Objective: Ensure NLP refiner improves output readability without changing intended meaning.

Expected Result: Raw token stream becomes fluent English with correct punctuation and preservation of domain-specific terms.

Test Steps:

1. Feed known raw outputs (fingerspelling, abbreviated grammar) to refiner.
2. Evaluate refined text against human-corrected ground truth (BLEU / human rating).

Outcome: Refinements are correct >85% of test cases (or as per project target).

Test Case 08: Emotion-conditioned TTS Output

Objective: Verify TTS synthesizes audio matching text & emotion.

Expected Result: Audio is intelligible and prosody reflects emotion label (e.g., happier pitch for "happy").

Test Steps:

1. Provide a refined sentence + emotion label to TTS.
2. Listen and check for differences across emotion labels

Edge Cases: Unsupported emotion label or missing voice model.

Outcome: If emotion unsupported, fallback to neutral voice and notify user.

Test Case 09: End-to-End Real-Time Recognition

Objective: Validate full pipeline from signing to emotional TTS in real time.

Expected Result: System outputs correct text and aligned emotional audio within latency budget.

Test Steps:

1. Sign several short sentences while system runs.
2. Observe live transcript, emotion indicator, and TTS playback.
3. Measure end-to-end latency for each utterance.

Outcome: Acceptable accuracy and latency; report any mismatches.

Test Case 10: Robustness to Lighting & Background

Objective: Check recognition stability across lighting and backgrounds.

Expected Result: Landmark detection and classification remain within acceptable degradation bounds.

Test Steps:

1. Test in bright, dim, backlit, and cluttered-background scenarios.
2. Evaluate drop in accuracy and landmark confidence scores.

Outcome: Report scenarios where performance is unacceptable and suggest mitigation (e.g., adaptive histogram equalization).

Test Case 11: Multiple Hands & Interference

Objective: Ensure single-signer operation or detect/reject multi-hand interference as defined.

Expected Result: System chooses primary hand (by proximity/handedness) or prompts user when multiple hands confuse the pipeline.

Test Steps:

1. Introduce a second hand into frame.
2. Observe how the system responds (ignore, switch, or alert).

Outcome: Behavior matches specification (e.g., alert "Multiple hands detected — place only one hand").

Test Case 12: Error Handling (Model Load Failure)

Objective: Ensure graceful failure if NN or DeepFace models fail to load.

Expected Result: User receives meaningful error and app offers fallback (e.g., disable TTS or emotion).

Test Steps:

1. Corrupt model files or simulate missing files.
2. Start app and observe behavior.

Outcome: App doesn't crash; displays error and disables affected features.

Test Case 13: Exit & Resource Release

Objective: Verify clean shutdown and resource release.

Expected Result: Pressing exit results in stopping camera, freeing memory/GPU, and terminating processes.

Test Steps:

1. Run system and press exit command.
2. Monitor system resources to confirm release.

Outcome: No lingering processes or locked camera, no memory leaks.

Test Case 14: User Acceptance Test (UAT) — Understandability

Objective: Evaluate end-user satisfaction with final outputs.

Expected Result: Users rate transcripts and TTS as accurate and helpful.

Test Steps:

1. Provide standard tasks to users.
2. Collect Likert-scale feedback and freeform comments.

Outcome: Collate feedback and actionable improvements.

4. Error and Exception Handling

Camera / Device Errors

if not cap.isOpened():

```
st.error("❌ Camera not found or permission denied")
st.info("💡 Try selecting a different camera device or check permissions")
st.session_state.camera_started = False
```

Mitigation: Multi-camera fallback, clear permission guidance, retry mechanisms

MediaPipe / Landmark Processing Failures

try:

```
results = hands.process(image_rgb)
```

except Exception as e:

```
st.warning(f"⚠️ Hand detection temporarily unavailable: {e}")
results = None
```

Recovery: Frame skipping, model reinitialization after consecutive failures, user notification

TensorFlow Lite Model Loading Errors

try:

```
self.interpreter = tf.lite.Interpreter(model_path=model_path)
self.interpreter.allocate_tensors()
```

except Exception as e:

```
st.error(f"❌ Failed to load ASL classification model: {e}")
st.error("💡 Check if model files are present and not corrupted")
```

Fallback: Disable gesture recognition, maintain emotion detection

Emotion Detection Failures

try:

```
results = DeepFace.analyze(aligned_face, actions=['emotion'],
                           enforce_detection=False, silent=True)
```

except Exception:

```
emotion_data = {"emotion": "neutral", "confidence": 0.0, "is_active": False}
```

Graceful Degradation: Continue with neutral emotion, maintain core ASL functionality

LM Studio / NLP Connection Failures

def refine_asl_buffer(buffer_string):

try:

```
response = client.chat.completions.create(...)
return process_response(response)
```

except Exception as e:

```
st.error(f"❌ NLP Refiner Error: {str(e)}")
st.error("💡 Make sure LM Studio is running on http://192.168.18.1:1234")
return {"refined_text": buffer_string, "error": str(e)}
```

Fallback: Return raw text with basic preprocessing, clear connection guidance

Piper TTS Synthesis Failures

Code Location: tts_manager.py audio generation

try:

```
audio_data = self.voice.synthesize(text, syn_config)
return self.generate_audio_bytes()
```

except Exception as e:

```
st.error(f'❌ TTS Generation Error: {str(e)}")  
st.error("💡 Try selecting a different voice or check voice files")  
return None
```

Recovery: Voice model switching, audio format fallbacks, text-only display option

MongoDB Connection Failures

try:

```
client = MongoClient(MONGODB_URI)  
client.admin.command('ping')  
return client
```

except Exception as e:

```
st.warning(f"⚠️ Database temporarily unavailable: {e}")  
return None
```

Continuity: Local session caching, delayed sync on reconnection, optional offline mode

Memory and Performance Monitoring

if fps < 15:

```
st.warning("⚠️ Performance degraded - consider reducing resolution or disabling features")
```

if len(st.session_state.letter_buffer) > BUFFER_MAX_SIZE:

```
st.session_state.letter_buffer.pop(0)
```

Adaptive Behavior: Automatic quality reduction, frame rate adaptation, memory cleanup

5. Limitations of the Solution

1. Emotion Recognition Reliability

- DeepFace pretrained models may not be equally accurate across ethnicities, occlusions (masks), or cultural expression differences. Emotions are inferred and may not reflect the signer's internal state.
- *Impact:* Mismatched prosody in TTS; consider showing emotion confidence and allowing user override.

2. Dataset Bias & Generalization

- NN accuracy depends heavily on training data. If training data lacks diversity in signers, lighting, skin tones, or signing styles, real-world performance will degrade.
- *Impact:* Lower recognition rates for underrepresented groups; mitigated by collecting diverse data and continuous model retraining.

3. **Single vs Multiple-Hand Signs**

- Current pipeline assumes single-hand gestures or simplifies multi-hand gestures. Two-handed signs (some ASL) are not fully supported.
- *Impact:* Reduced linguistic coverage; future versions should extend to multi-hand temporal modeling.

4. **Real-time Latency on Low-end Devices**

- Landmark extraction, NN inference, DeepFace face analysis, NLP, and TTS are computationally expensive. On low-end CPUs (or no GPU), the system may stutter or introduce unacceptable latency.
- *Impact:* Poor user experience; mitigations: lighter models, frame skipping, server-side inference.

5. **Continuous Sentence Segmentation Complexity**

- Mapping continuous sign streams to syntactically correct English (with grammar and punctuation) is a hard NLP problem. Fingerspelling, topicalization in sign languages, and non-manual markers are challenging to convert reliably.
- *Impact:* Semantic mismatches; include user correction UX and conservative refinements.

6. **Privacy & Sensitivity**

- Capturing video and facial emotion data raises privacy concerns. Storing or transmitting such data must follow privacy best practices (consent, encryption, retention limits).
- *Impact:* Need clear privacy policy and opt-in controls.

7. **Background & Clothing Confounds**

- Similar-colored backgrounds or skin-tone/clothing contrasts can affect landmark detection if not handled by MediaPipe preprocessing.

- *Impact:* False landmarks; advise neutral backgrounds or implement adaptive background subtraction.

8. TTS Emotion Mapping Granularity

- Available TTS voices and emotion controls may be limited (categorical emotions only) and not capture nuanced affect. Some emotions like “sarcastic” or “subtle annoyance” may not be supported.
- *Impact:* User-perceived mismatch between detected emotion and spoken prosody.

9. Calibration Requirement

- Although MediaPipe reduces need for color calibration, the system may still require tuning (camera position, lighting). Initial setup can be a barrier for quick adoption.
- *Impact:* Provide guided calibration wizard and default presets.

10. Legal / Accessibility Considerations

- System should be treated as an assistive tool, not a substitute for certified interpreters in high-stakes contexts (medical, legal).
- *Impact:* Document intended use and disclaimers.

6. Testing Images

A, M N, S have similar appearance but correct detection by our system:

Controls

Camera Device

0

Width

1280

Height

720

☒ Enable Emotion Detection

☒ Enable Translation

Translation Settings

Target Language

German

☒ Enable Text-to-Speech

Voice Model

Bryce (Male - Medium)

Speech Speed

1.00

ASL + Emotion Detection

Real-time ASL letter recognition with background emotion analysis.

FPS: 12.95

Mood: NEUTRAL

Right: A



Detection Results

Predicted Letter

A

Holding 'A'

Overall Tone

Neutral

88% Confidence

FPS

12.9

Letter Buffer

Sequence

Controls

Camera Device

0

Width

1280

Height

720

☒ Enable Emotion Detection

☒ Enable Translation

Translation Settings

Target Language

German

☒ Enable Text-to-Speech

Voice Model

Bryce (Male - Medium)

Speech Speed

1.00

Min Detection Confidence

ASL + Emotion Detection

Real-time ASL letter recognition with background emotion analysis.

FPS: 14.08

Mood: NEUTRAL

Right: M



Detection Results

Predicted Letter

M

Holding 'M'

Overall Tone

Neutral

95% Confidence

FPS

14.1

Letter Buffer

Sequence

AABBAAELLCAAABBBCCDDDEEE
FFGGGHHIIIIZZZKKKJJJLLSMM

Controls

Camera Device

0

Width

1280

Height

720

Enable Emotion Detection

Enable Translation

Translation Settings

Target Language

German

Enable Text-to-Speech

Voice Model

Bryce (Male - Medium)

Speech Speed

1.00

Min Detection Confidence

ASL + Emotion Detection

Real-time ASL letter recognition with background emotion analysis.

FPS: 13.92

Mood: NEUTRAL

Right: N

Detection Results

Predicted Letter

N

Holding 'N'

Overall Tone

Neutral

84% Confidence

FPS

13.9

Letter Buffer

Sequence

AABBBAEELLCAAABBBCCDDDEE
EFFFFGGHHIIIIZZKKKJJLLSM

Controls

Camera Device

0

Width

1280

Height

720

Enable Emotion Detection

Enable Translation

Translation Settings

Target Language

German

Enable Text-to-Speech

Voice Model

Bryce (Male - Medium)

Speech Speed

1.00

Min Detection Confidence

ASL + Emotion Detection

Real-time ASL letter recognition with background emotion analysis.

FPS: 12.41

Mood: NEUTRAL

Right: S

Detection Results

Predicted Letter

S

Holding 'S'

Overall Tone

Neutral

98% Confidence

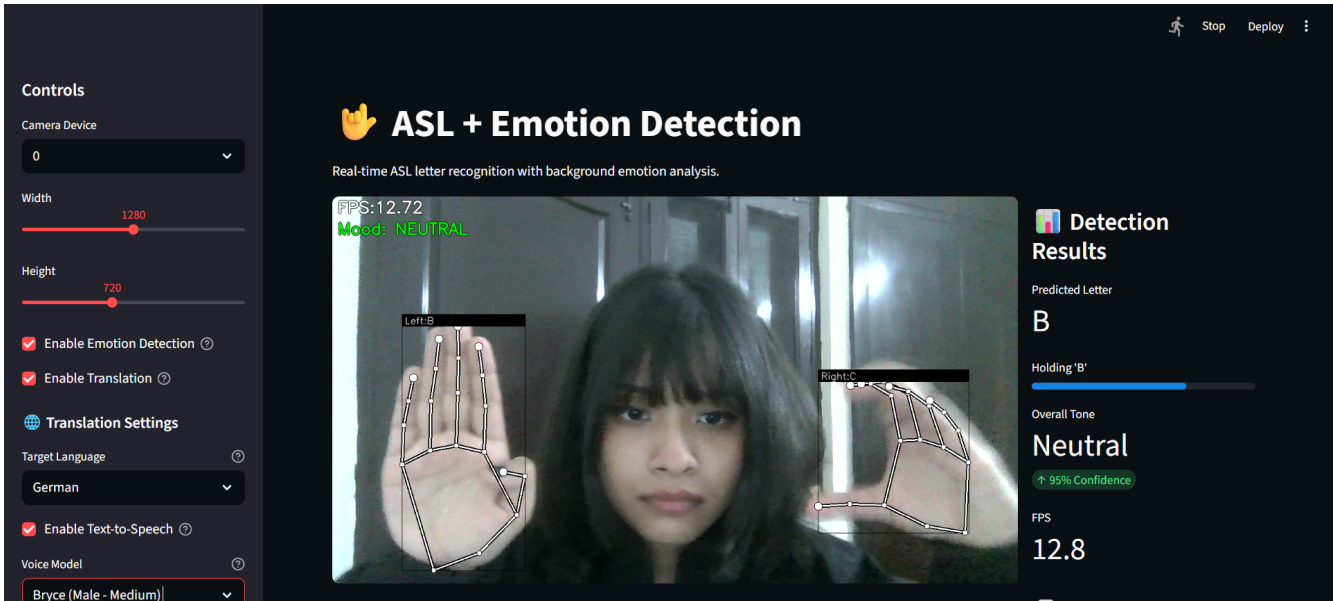
FPS

12.5

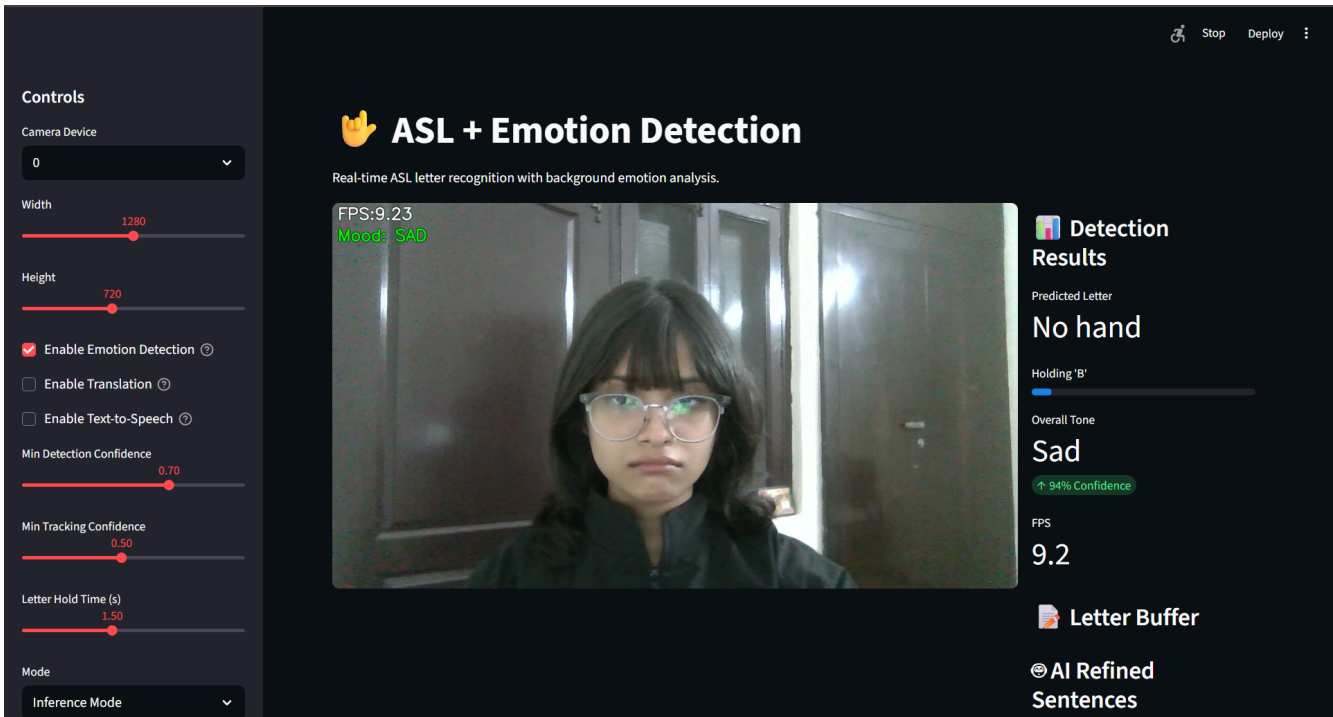
Letter Buffer

Sequence

Detecting both hands even if there is motion blur:



Emotion detection with ASL:



Controls

Camera Device

0

Width

1280

Height

720

☒ Enable Emotion Detection ⓘ

☒ Enable Translation ⓘ

Translation Settings

Target Language ⓘ

German

☒ Enable Text-to-Speech ⓘ

Voice Model ⓘ

Bryce (Male - Medium)

Speech Speed

1.00

👋

ASL + Emotion Detection

Real-time ASL letter recognition with background emotion analysis.

FPS: 14.84

Mood: HAPPY

Left: C



Detection Results

Predicted Letter

C

Holding 'C'

Overall Tone

Happy

↑ 99% Confidence

FPS

14.8

Letter Buffer

Sequence

AABBBAAELLCAAABBBCCC

Controls

Camera Device

0

Width

1280

Height

720

☒ Enable Emotion Detection ⓘ

☐ Enable Translation ⓘ

☐ Enable Text-to-Speech ⓘ

Min Detection Confidence

0.70

Min Tracking Confidence

0.50

Letter Hold Time (s)

1.50

Mode

Inference Mode

👉 ASL + Emotion Detection

Real-time ASL letter recognition with background emotion analysis.

FPS:5.74

Mood: SURPRISE



Detection Results

Predicted Letter

E

Holding 'E'

Overall Tone

Surprise

↑ 76% Confidence

FPS

5.7

Letter Buffer

Sequence

AABBBAA

Controls

Camera Device

0

Width

1280

Height

720

☒ Enable Emotion Detection

☐ Enable Translation

☐ Enable Text-to-Speech

Min Detection Confidence

0.70

Min Tracking Confidence

0.50

Letter Hold Time (s)

1.50

Mode

Inference Mode

👉

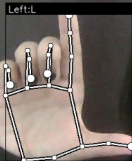
ASL + Emotion Detection

Real-time ASL letter recognition with background emotion analysis.

FPS:8.66

Mood: FEAR

Left:L





Detection Results

Predicted Letter

L

Holding 'L'

Overall Tone

Fear

↑ 45% Confidence

FPS

8.7

Letter Buffer

Sequence

AABBBAAEL

Controls

Camera Device

0

Width

1280

Height

720

☒ Enable Emotion Detection ⓘ

☒ Enable Translation ⓘ

Translation Settings

Target Language ⓘ

German

☒ Enable Text-to-Speech ⓘ

Voice Model ⓘ

Bryce (Male - Medium)

Speech Speed

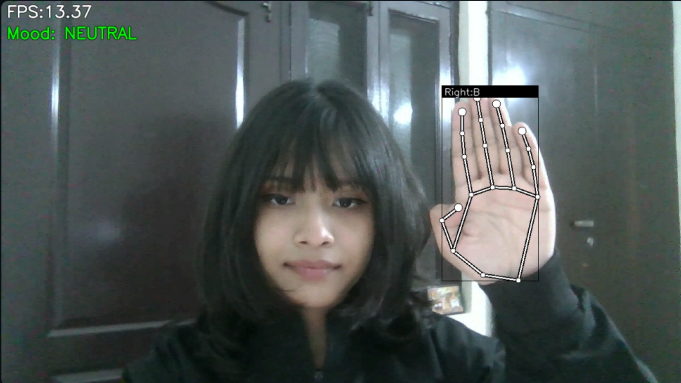
1.00

Min Detection Confidence

👋 ASL + Emotion Detection

Real-time ASL letter recognition with background emotion analysis.

FPS: 13.37
Mood: NEUTRAL



Detection Results

Predicted Letter

B

Holding 'B'

Overall Tone

Neutral

↑ 99% Confidence

FPS

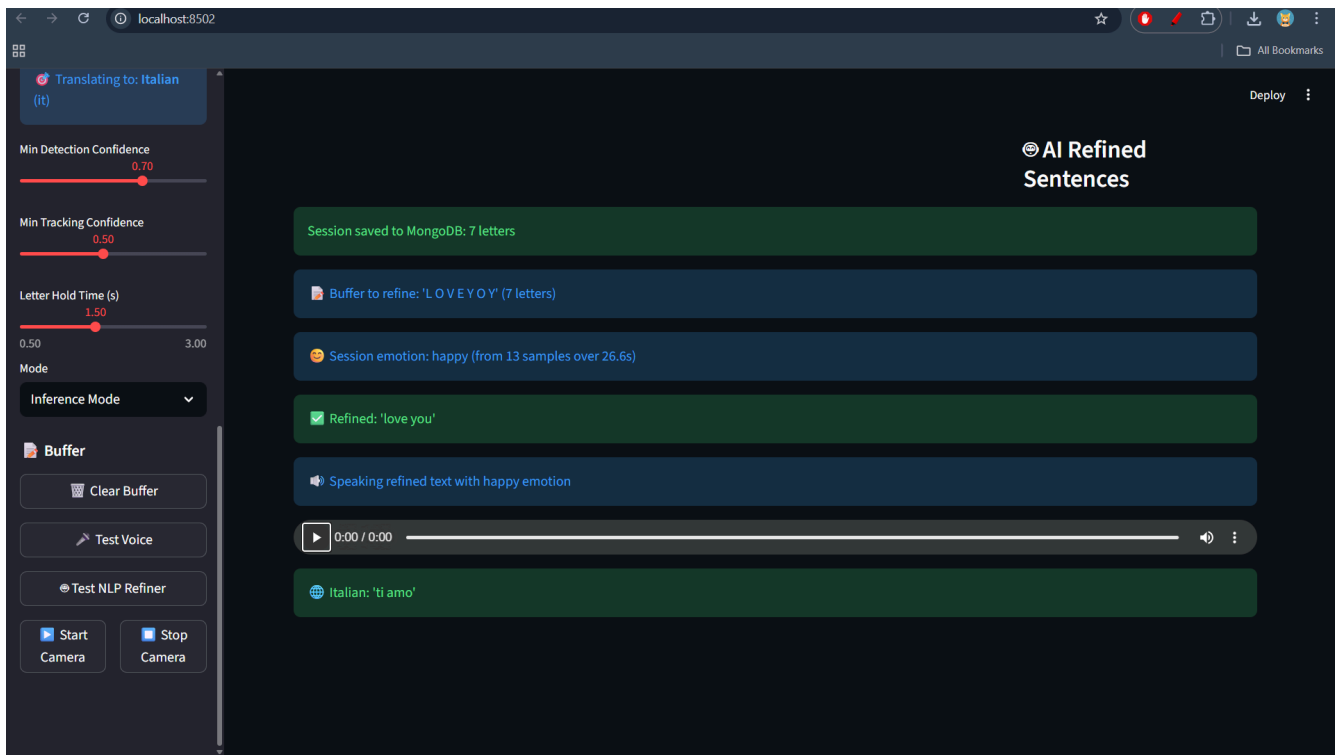
13.4

Letter Buffer

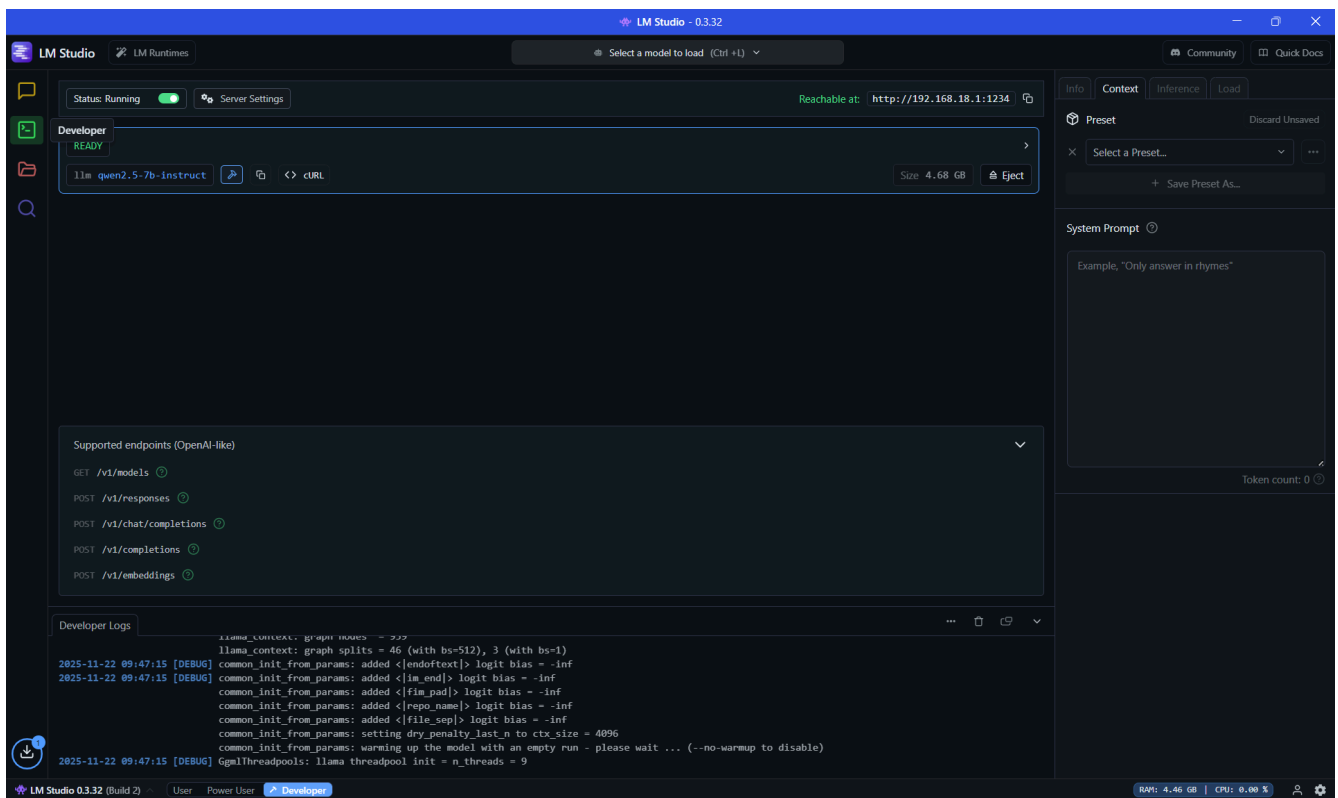
Sequence

AABBBAAELLCAABB

Text-to-Speech with emotion detection



Loading Qwen2.5 7B Instruct



CHAPTER 6: FINDINGS, CONCLUSION AND FUTURE WORK

1. Findings

1. Alphabet-level Gesture Recognition:

The system accurately recognized static sign gestures corresponding to the English alphabet.

The gesture recognition module performed well when the user signed within the camera frame and maintained consistent hand orientation. However, since only alphabet-level signs were supported, speed and clarity of signing played a major role in prediction accuracy.

2. Emotion Detection Performance:

The emotion recognition module, built using the DeepFace pretrained model, successfully detected the signer's facial emotion in real time. Emotions such as happy, sad, angry, neutral, surprised, and fearful were recognized as reliable.

3. Lighting and Background Sensitivity:

Both gesture recognition and emotion recognition were sensitive to lighting variations. Bright spots, shadows, or cluttered backgrounds reduced the accuracy of gesture segmentation and facial recognition. Controlled lighting conditions produced the most stable results.

4. Context Generation Using NLP:

Since the gesture model captured only alphabets, the raw gesture stream was typically noisy and lacked grammar. The NLP module effectively transformed these alphabet sequences into meaningful English sentences by applying spelling correction, grammar correction, and context-aware restructuring.

5. Emotion-Aware TTS Output:

The Piper TTS engine successfully converted corrected English sentences into speech. The emotional tone of the output changed based on the emotion detected by DeepFace. Although Piper currently supports only English voices, it was able to express basic prosody variations aligned with the detected emotion.

6. Real-Time Performance:

The application performed smoothly on mid-range GPUs and capable CPUs. However, lower-end devices showed reduced frame rates, causing slight delays in gesture prediction and emotion recognition.

7. Model Generalization Limitations:

Differences in signing speed, hand angle, skin tone, and camera distance sometimes affected

gesture recognition. Similarly, emotion detection accuracy reduced when facial occlusions (mask, glasses) were present or when the user's face was partially outside the frame.

2. Conclusions

1. Integrated Multimodal System Successfully Demonstrated:

The project successfully combined gesture recognition, emotion detection, NLP-based sentence correction, and TTS synthesis into a functional sign-to-text communication system. This demonstrated the feasibility of multimodal translation for sign language users.

2. Context-Aware Sign Language Translation Achieved:

Unlike standard alphabet detectors, the system was able to form meaningful, grammatically correct English sentences using NLP, significantly improving communication clarity.

3. Emotion-Enhanced Speech Output:

By integrating facial emotion recognition and TTS prosody control, the system offered a richer and more expressive communication experience. This allowed the output speech to convey not only the signer's words but also their emotional intent.

4. Practicality with Limitations:

While the prototype works effectively under controlled conditions, real-world usability is limited by factors such as lighting, gesture vocabulary size, device capability, and TTS voice constraints. Nevertheless, the system establishes a strong foundation for future expansions.

3. Future Work

1. Expand Gesture Vocabulary Beyond Alphabets:

Future enhancements should include recognition of full words and continuous sign phrases rather than only static alphabet signs. Incorporating deep learning models such as Mediapipe Holistic, 3D CNNs, or Transformer-based gesture encoders would enable full sign language translation.

2. Improved NLP for Better Contextual Understanding:

Use larger language models or fine-tuned transformers to generate more context-aware sentences, recognize semantic patterns, and handle longer alphabet input streams more intelligently.

3. More Advanced Emotion Modeling:

Improve emotion recognition using video-based models instead of image-based detection. This

would allow recognition of subtle emotional transitions and micro-expressions for more expressive TTS output.

4. Enhanced TTS Capabilities:

- Add support for **more voices** so users can personalize the output.
- Integrate **higher-quality paid TTS engines** (e.g., ElevenLabs, Amazon Polly, Azure TTS) for natural-sounding emotional expression.
- Incorporate **multilingual TTS**, allowing translation into multiple languages instead of English only.

5. User Adaptation and Personalization:

Enable the system to learn the signing style of a specific user over time, improving recognition accuracy for frequently used gestures.

6. Robustness to Real-World Conditions:

Improve performance under varying lighting conditions, cluttered backgrounds, and different camera qualities by integrating advanced preprocessing and adaptive thresholding techniques.

7. Mobile and Lightweight Deployment:

Optimize the model for mobile or edge devices to make the system portable and accessible to a wider audience.

8. Emotion-Integrated Continuous Sign Translation:

In future versions, combine continuous sign recognition with continuous emotion tracking to produce real-time, emotionally expressive speech output for complex, multi-sentence communication.

REFERENCES

1. [1] Zhang, F., Bazarevsky, V., Vakunov, A., Tkachenka, A., Sung, G., Chang, C. L., & Grundmann, M. (2020). Mediapipe hands: On-device real-time hand tracking. *arXiv preprint arXiv:2006.10214*.
2. [2] Bazarevsky, V., Kartynnik, Y., Vakunov, A., Raveendran, K., & Grundmann, M. (2019). Blazeface: Sub-millisecond neural face detection on mobile gpus. *arXiv preprint arXiv:1907.05047*.
3. [3] Culjak, I., Abram, D., Pribanic, T., Dzapo, H., & Cifrek, M. (2012, May). A brief introduction to OpenCV. In *2012 proceedings of the 35th international convention MIPRO* (pp. 1725-1730). IEEE.
4. [4] Venkatesan, R., Shirly, S., Selvarathi, M., & Jebaseeli, T. J. (2023). Human emotion detection using DeepFace and artificial intelligence. *Engineering Proceedings*, 59(1), 37.
5. [5] Bryant, C., Yuan, Z., Qorib, M. R., Cao, H., Ng, H. T., & Briscoe, T. (2023). Grammatical error correction: A survey of the state of the art. *Computational Linguistics*, 49(3), 643-701.
6. OpenCV Official Documentation – <https://docs.opencv.org/4.x/>
7. MediaPipe Official Documentation – <https://developers.google.com/mediapipe/>
8. TensorFlow Official Documentation – https://www.tensorflow.org/api_docs
9. PyTorch Official Documentation – <https://pytorch.org/docs/stable/index.html>
10. Piper TTS Documentation – TTS 0.22.0 documentation
11. Qwen LLM / LM Studio – <https://www.lmstudio.ai>